Identifiers

An identifier is defined as a letter followed by zero or more letters, digits, or single underscores:

```
ID ::= identifier_letter ( ('_')? letter_or_digit)*
letter_or_digit ::= identifier_letter | digit
identifier_letter ::= ('A'..'Z' | 'a'..'z')
digit ::= (0..9)
```

Examples of valid identifiers: `Count`, `X`, `Get_Symbol`, `Ethelyn`, `Snobol_4`, `X1`, `Page_Count`

Identifiers are case-insensitive.

Boolean values and numeric literals are defined as follows:

```
Literal :: = Boolean_literal | Integer_literal | Real_literal
Integer_literal ::= decimal_integer_literal
Real_literal ::= decimal_real_literal
decimal_integer_literal ::= -('')? numeral
decimal_real_literal ::= -('')? numeral '.' numeral
numeral ::= digit*
```

Corresponding examples of `Boolean_literal`: `true` and `false`

Corresponding examples of `Integer_literal`: 1, 42, -1337

Corresponding examples of `Real_literals`: 3.1415, 1.6180, 0.001

String elements are defined by the following syntax:

```
STRING ::= "(string_element)*"
string_element ::= "" | non_quotation_mark_graphic_character
```

Data Types

Primitive Types:
`real`, `bool`, `int`, `enumeration`

在使用`real`, `bool`, `int`这三种表达时不可直接使用，必须继承自`Base_Types`包，如下代码所示：

```
-- Type Aliases

data int extends base_types::Integer
end int;

data real extends base_types::Float
end real;

data bool extends base_types::Boolean
end bool;
```

AGREE retrieves AADL features of the following types from the `Base_Types` package in the AADL plugin repository:

1. Base_Types::Boolean
2. Base_Types::Integer
    - Base_Types::Integer_8
    - Base_Types::Integer_16
    - Base_Types::Integer_32
    - Base_Types::Integer_64
    - Base_Types::Unsigned_8
    - Base_Types::Unsigned_16
    - Base_Types::Unsigned_32
    - Base_Types::Unsigned_64
3. Base_Types::Float
    - Base_Types::Float_32
    - Base_Types::Float_64

In AGREE, all integer and unsigned types with specific bit sizes are approximated using unbounded integers, and all floating-point numbers are approximated using rational numbers:

- Base_Types::Boolean is mapped to the AGREE primitive type "bool"
- All integer and unsigned types of various sizes in Base_Types are mapped to the AGREE primitive type "int"
- Floating-point types in Base_Types are mapped to the AGREE primitive type "real"

Composite Types: `Record`

Example code:

```
data Alarm_Outputs
end Alarm_Outputs;

data implementation Alarm_Outputs.Impl
  subcomponents
    Is_Audio_Disabled : data Base_Types::Boolean;
    Notification_Message : data Base_Types::Integer;
    Log_Message_ID : data Base_Types::Integer;
end Alarm_Outputs.Impl;
```

Use a dot (.) to reference specific members in a `record` type.

Example code：

```
Alarm.Is_Audio_Disabled => Alarm.Log_Message_ID > 3;
```

*Enumeration Types

Enumeration types can be specified directly in the AGREE annex or in AADL.

To define an enumeration type in the AADL data model, two key properties are required:

- Data_Representation => Enum： Declares that this data type is an enumeration type;
- Enumerators => ("Value 1", "Value 2", ...): Specifies all optional values included in the enumeration type.

AADL specification example code:

```
data Primary_Color
  properties
    Data_Model::Data_Representation => Enum;
    Data_Model::Enumerators => ("Red", "Green", "Blue");
end Primary_Color;

data Pixel
end Pixel;

data implementation Pixel.Impl
  subcomponents
    color : data Primary_Color;
    x_pos : data Base_Types::Integer;
    y_pos : data Base_Types::Integer;
end Pixel.Impl;
```

AGREE specification example code：

```
enum card_suit = { spade, heart, club, diamond };
eq color : card_suit;
```

Subrange Types

They can be declared using the integer and real range properties of the AADL data model, which is suitable for defining reusable types.

Example code:

```
data Step_Count extends Base_Types::Integer
  properties
    Data_Model::Integer_Range => 1 .. 12;
end Step_Count;

data Audio_Volume_Level extends Base_Types::Float
  properties
    Data_Model::Real_Range => 0.0 .. 11.0;
end Audio_Volume_Level;
```

In AGREE, you can define ranges for integer (`int`) or real (`real`) types, and the restricted types are subrange types.

Example code:

```
eq n : int[1,3];
eq x : real[0.0, 10.0];
```

Array Types

AGREE array indices start from 1 (consistent with AADL).

Array types can be used as parameters for nodes or functions.

AGREE array types are specified by appending array dimensions directly after the base type.

Example code:

```
eq x : int[3];
eq xx : int[3][2];
eq arr : int[3];  -- Define an array arr containing 3 integer elements
```

Assigning initial values when defining an array: Values must be provided for **all elements** of the array without omission. Wrap element values with `[|` and `|]`, and separate elements with commas.

```
-- Declare an integer array a of length 3 and assign values using an array expression

eq a : int[3] = [| 1, 2, 3 |];
```

Updating an array: Use the `:=` symbol to modify some or all elements of an existing array while retaining the original values of unmodified elements. Ensure the data type remains consistent before and after modification, and finally return a new updated array.

```
-- 1. First declare and initialize the original array arr
eq arr : int[3] = [| 1, 2, 3 |];

-- 2. Modify elements at indices 2 and 3 of arr using an array update expression and assign the result to updated_arr
eq updated_arr : int[3] = arr [| 2 := 6, 3 := 12 |];
```

Array access:

```
eq n : int = arr[1];  -- Access the first element of arr and assign it to variable n
```

AGREE recognizes arrays defined using AADL syntax in AADL data implementations, but AADL arrays must have fixed dimensions.

Example code:

```
data Location end Location;
data implementation Location.Impl
  subcomponents
    latitude: data Base_Types::Real;
    longitude: data Base_Types::Real;
    altitude: data Base_Types::Real;
end Location.Impl;

data Track end Track;
data implementation Track.Impl
  subcomponents
    waypoints : data Location.Impl[4];
end Track.Impl;
```

AGREE annex

Basic form:

```
annex agree {**
    -- agree declarations here...
**};
```

It can be placed in AADL components such as systems, processes, and threads.

It can also be placed at the top level of a package as a reusable "specification library", where defined nodes, constants, etc., can be referenced by other components.

Example code:

```
eq x1 : int = Agree_Common.Counter(0, 1, prev(x1 = 9, false));
```

*Assume Statements

Assumptions about inputs, assuming certain preconditions are true, serving as the basis for analyzing or verifying component behavior.

Structure format:

```
assume [identifier (optional)] ["description string"] : boolean_expression;
```

Example code:

Note: `A001`: Optional identifier for the assume statement, which must be unique in the local namespace.

```
assume A001 "System input domain" : Input < 10;
```

During verification:

- For the current component: When verifying the implementation of this component, these assumptions are directly trusted to be true (i.e., `Input < 10` is assumed to hold by default) and do not need to be proven.
- For subcomponents: The assumptions of the subcomponents themselves need to be "proven" to hold — the basis for the proof includes the assumptions of the current component and the "guarantee conditions" of other subcomponents.

Guarantee Statements

Constraints that a component must always maintain on the premise that all assumption conditions are satisfied.

`guarantee` is used to specify that as long as all assumption conditions (`assume`) of the component always hold, the component will definitely satisfy the constraint conditions.

Structure format:

```
guarantee [identifier (optional)] ["description string"] : boolean_expression;
```

Example code:

Note: `G001`: Optional identifier for the guarantee statement, which must be unique in the local namespace.

```
guarantee G001 "System output range" : Output < 50;
```

When verifying component implementation, guarantee statements need to be proven to hold. The basis for the proof includes:

- The component's own assumption conditions (`assume`);
- The guarantee conditions defined in the contracts of its internal subcomponents (subcomponents' `guarantee`).

Equation Statements

Define local variables in the AGREE clause, which do not appear in the interface of the architecture model.

Name complex expressions for easy reuse.

1. Explicit definition:

   Structure format:

   ```
   eq variable_name : type = expression;
   ```

   Example code:

   ```
   eq ctr : int = prev(ctr + 1, 0);
   ```

   Meaning: The current value of `ctr` is equal to the previous value of `ctr + 1`, with an initial value of 0.

2. Implicit definition:

   ```
   eq temp : real;  // Only declare the variable without assigning a value
   guarantee "Temperature range" : temp > 0 and temp < 100;
   ```

   Only declare the variable without directly assigning a value; instead, constrain it through other statements (`assume`, `assert`, `guarantee`, etc.).

3. Define multiple variables simultaneously:

   ```
   eq a : int, b : real;  // Declare two variables simultaneously (implicit definition)
   eq x : int, y : int = Add(3, 5);  // Assume the Add node returns two values, assigned to x and y respectively
   ```

   Suitable for associating with the results of a node that returns multiple values.

Assign Statements

Specialized for assigning values to component output ports, ensuring a clear relationship between outputs and inputs/local variables.

Directly assign values to variables.

Structure format:

```
assign target_variable = expression;
```

Example code:

```
assign output_voltage = input_voltage + compensation_voltage;
```

Logically equivalent to the `assert` statement, but with an additional check: to avoid duplicate assignments to output ports.

Property Statements

Used to name a `boolean` expression for easy reuse elsewhere. Functions the same as equation statements of `boolean` type.

Structure format:

```
property property_name = boolean_expression;
```

Example code:

```
property not_system_start_implies_mode_0 = not(OP_CMD_IN.System_Start) →
(GPCA_SW_OUT.Current_System_Mode = 0);
```

Equivalent to defining and assigning a boolean-type variable:

```
eq not_system_start_implies_mode_0 : bool = not(OP_CMD_IN.System_Start) →
(GPCA_SW_OUT.Current_System_Mode = 0);
```

Constant Statements

Assign a fixed value to a name.

Used in exactly the same way as variables defined by equation statements (`eq`).

Structure format:

```
const constant_name: type = value;
```

Example code:

```
const ADS_MAX_PITCH_DELTA: real = 1.0;
```

Node Definitions

Nodes are used to define reusable logical units, similar to functions but can **maintain internal state** (i.e., variable values change with time steps). They can accept multiple inputs and return multiple outputs, suitable for encapsulating common functions (such as counters, adders, etc.) and can be reused multiple times in the model.

Basic structure:

- Input parameters
- Return values
- Internal logic (statements between `let` and `tel`, defining how return values are calculated)

Node definition structure format:

```
node node_name(input_parameter1: type1, input_parameter2: type2, ...) returns (output_parameter1: type1, ...);
let
  output_parameter1 = expression1;
  output_parameter2 = expression2;
  // (Optional) Local variable definitions and logic
tel;
```

Node definition example code:

```
node Counter(init: int, incr: int, reset: bool) returns (count: int);
let
  count = if reset then init else prev(count, init) + incr;
tel;
```

Node call: Defined nodes can be called in equation statements (`eq`), passing parameters and receiving return values. If a node returns multiple values, variables separated by commas are required to receive them.

Call example code:

```
eq x1 : int = Counter(0, 1, prev(x1 = 9, false));
eq x2 : int = Counter(1, prev(x2, 0), false);
```

Complex nodes:

Nodes can contain multiple inputs and multiple outputs.

They can also define internal local variables through `var`,

and even nest calls to other nodes.

Example code:

```
node ADD1(a : bool, b : bool, carry_in : bool) returns (out : bool, carry_out : bool);
let
  out = (a <> b) <> carry_in;
  carry_out = (a and b) or (a and carry_in) or (b and carry_in);
tel;

node ADD4(a0,a1,a2,a3 : bool, b0,b1,b2,b3 : bool) returns (s0,s1,s2,s3 : bool, carry_out : bool);
var  // Local variables: intermediate carries
  c0 : bool; c1 : bool; c2 : bool; c3 : bool;
let
  s0,c0 = ADD1(a0, b0, false);
  s1,c1 = ADD1(a1, b1, c0);
  s2,c2 = ADD1(a2, b2, c1);
  s3,c3 = ADD1(a3, b3, c2);
  carry_out = c3;
tel;
```

Record Definitions

Composite data types that can combine multiple fields of different types into a single unit.

Can be directly defined in the AGREE annex:

Definition structure format:

```
type record_type_name = struct { field1: type1, field2: type2, ..., fieldn: typen };
```

Example code:

Declare a variable bar of type foo:

```
annex AGREE {**
    type foo = struct {a : bool, b : int};
    eq bar : foo;
**};
```

Construct a record value of type foo and assign it to bar2:

```
annex AGREE {**
    type foo = struct {a : bool, b : int};
    eq bar2 : foo = foo { a = true; b = 6 };
**};
```

Modify some or all fields of an existing record while keeping other field values unchanged.

Syntax: `original_record { field_name := new_value; ... }`

```
annex AGREE {**
    type foo = struct {a : bool, b : int};
    eq bar2 : foo = foo { a = true; b = 6 };  -- Initial record

    eq updated_bar : foo = bar2 {a := false};
    -- The value of updated_bar is {a = false; b = 6}
**};
```

*Real-time Patterns

Specification patterns used to describe real-time system behavior constraints, which can replace ordinary expressions in `assume`, `guarantee`, or `assert` statements.

Real-time patterns rely on the following built-in elements to describe time:

1. `time` variable: A reserved variable representing the current time, which appears in counterexamples generated by the tool.
2. Time functions (with a boolean variable `id` as the parameter:)

```
id
```

- `timeof(id)`: Returns the `time` value when `id` was last `true`; returns `-1.0` if it has never been `true`.
- `timerise(id)`: Returns the `time` value when `id` last transitioned from `false` to `true` (rising edge); returns `-1.0` if it has never occurred.
- `timefall(id)`: Returns the `time` value when `id` last transitioned from `true` to `false` (falling edge); returns `-1.0` if it has never occurred.

AGREE supports three core real-time patterns:

`WhenStatement`

(When... pattern)

Description: When condition A holds within a certain time interval, condition B must occur within another time interval.

Structure format:

```
when condition A holds during [time_interval1] condition B (exclusively)? occurs during [time_interval2]
```

`WheneverStatement`

(Whenever... pattern)

Description: Whenever condition A occurs, condition B must satisfy a certain time relationship.

Structure format:

```
// Whenever A occurs, B must occur within the time interval
whenever condition A occurs condition B (exclusively)? (occur/occurs) during [time_interval]

// Whenever A occurs, B must hold continuously within the time interval
whenever condition A occurs condition B (exclusively)? holds during [time_interval]

// Whenever A occurs, it means B must be satisfied within the time interval
whenever condition A occurs condition A implies condition B (exclusively)? during [time_interval]
```

`RealTimeStatement`

(Real-time condition pattern)

Description: Time constraints for periodic or sporadic events.

Structure format:

```
// Periodic events: Condition A occurs every X time, with optional jitter
condition condition A occurs each time_interval (with jitter jitter_value)?

// Sporadic events: The interval between occurrences of condition A is not less than X, with optional jitter
condition condition A occurs sporadic with IAT minimum_interval (with jitter jitter_value)?

// IAT (Inter-Arrival Time): The minimum interval between event occurrences
```

Time Interval

Represented by square brackets, in the format `[start_time, end_time]`

Assert Statements

Assert statements (`assert`) are used to define "statements considered facts" (axioms) in the model.

These statements are treated as "established facts" of system or environmental behavior, used to assist in proving other constraints (such as guarantee statements).

Used in the same way as `assign`, and it is not recommended to use unless necessary.

Difference from `assign`: AGREE will never verify whether the assertion itself holds (while the `assume` of subcomponents needs to be proven). `assert` fully relies on manual assurance of its correctness.

Lemma Statements

Assist the model checker in verification, mainly used to enhance proof capabilities.

In AGREE, verification usually adopts k-induction, which can automatically prove system properties in most cases. However, sometimes certain properties are "logically true but too weak to be directly proven by induction". At this time, lemmas, as "strengthened properties", can provide additional basis for the proof and make the core properties provable.

Structure format:

```
lemma [identifier (optional)] ["description string"] : boolean_expression;
```

Example code:

```
lemma L001 "drug flow lemma" : (not drug_flow_stopped) => spo2_never_below_thresh;
```

Similar to guarantee statements `guarantee`: Must be proven true by AGREE, and the basis for the proof is also the component's assumptions and the guarantees of subcomponents.

Different from guarantee statements `guarantee`: Lemmas are not exposed to higher-level abstract verification. That is, the lemmas of subcomponents are not used to prove the guarantees of other subcomponents or the system level, but only serve the proof of `guarantee` within the current component.

*Reachable Statements

Auxiliary statements used to check for "vacuity" in guarantee statements (`guarantee`) or lemma statements (`lemma`).

Vacuity: If a guarantee statement is "if A then B" (`A → B`), but A can never occur in any case (A is always false), then although the guarantee statement is logically "always true", it is actually meaningless.

Structure format:

```
reachable "identifier (optional)" "description string" : boolean_expression;
```

Example code:

```
reachable R001 "drug flow might not be stopped" : (not drug_flow_stopped);
```

Similar to lemmas (`lemma`), reachability statements do not participate in the composite contract analysis of components and are only used to check for vacuity.

*Linearization Definitions

In formal verification of model checking, nonlinear expressions such as square `y²`, sine `sin(y)`, and exponential `exp(y)` cannot be processed by the tool, so the differential idea is used.

Structure format:

```
linearization name(input_parameter: type) over [interval_start ..
interval_end] within precision : nonlinear_expression;
```

Example code:

```
linearization sq (y : real) over [-1.0 .. 1.0] within 0.01 : y^2.0;
```

Syntax Element	Description
`linearization`	Keyword declaring this is a linearization definition.
`sq`	Name of the linearization (customizable for subsequent references).
`(y : real)`	Input parameter: A single input `y` of real type (`real`) — currently AGREE only supports linearization with a single real input.
`over [-1.0 .. 1.0]`	Input interval: Specifies the valid range of the linearization — only takes effect when `y` is between `-1.0` and `1.0`.
`within 0.01`	Precision requirement: The error of the linear approximation cannot exceed `0.01` (i.e., the maximum distance between the linear segment and the nonlinear curve ≤ 0.01).
`: y^2.0`	Target nonlinear expression: The expression to be linearized, here is "the square of `y`" (`y²`).

Supported nonlinear expressions:

- Basic operations: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `^` (exponentiation, e.g., `y^2` represents square);
- Transcendental functions: `abs` (absolute value), `sin` / `cos` / `tan` (trigonometric functions), `asin` / `acos` / `atan` (inverse trigonometric functions), `exp` (exponential), `log` / `log10` (logarithm), `sqrt` (square root), etc.

Lemma expressions call defined linearization expressions:

```
eq y : real;
lemma "sq() range positive" : sq(y) <= -0.10;
```

Uninterpreted Functions

Suitable for scenarios where the specific implementation is not concerned and only the logical relationship before and after function calls needs to be verified, which can reduce verification complexity.

A function type that declares input and output types without defining specific implementation logic.

The only constraint is "the same input must correspond to the same output: if `x = y`, then `f(x) = f(y)`".

Declaration format:

```
uninterpreted function_name(parameter1: type, parameter2: type, ...) :
return_type;
```

Example code:

```
uninterpreted sincos(x: real) : real;
```

Calling method:

```
eq result : real = sincos(3.14);
```

Expressions

Composition:

1. Relational operators
2. Identifiers
    - `QID`: Qualified identifiers like `ID::ID`, referencing "which element under which module"
    - `NestedDotID`: Nested identifiers with dots, such as `a.b.c`
    - `Literal`: Literals, including boolean values, integers, and real numbers

Expression forms:

- Basic elements: Literals, `ID`, `QID`, `NestedDotId`
- Function calls: `ID(Expr_List)`
- Special functions:
  - `pre(Expr)`: Represents the value of the previous state
  - `prev(Expr, Expr)`: Same as `pre`, but returns the second parameter (default value) in the initial state without a previous step
  - `event(NestedDotID)`: Represents an event
  - `floor(Expr)`: Rounding down
  - `real(Expr)`: Conversion to real number
  - `Get_Property(Expr, AADL_Property)`: Get property
  - `this(.NestedDotId)?`: Property of the current object
- Parenthesized expressions: `(Expr)`
- Record and array update expressions: RecordUpdateExpr, ArrayUpdateExpr
- Unary minus operator: `-` or `not` expressions
- Arithmetic operators: `+`, `-`, `*`, `/`, `div`, `mod`
- Relational operations: Expression + RelateOp + Expression
- Logical operations: `and`, `or`
- Implication relationships: `->`, `=>`, `<=>`
- Conditional expressions: `if Expr then Expr else Expr`

Operator precedence (from lowest to highest):

1. `->` | `=>` | `<=>`
2. `or`
3. `and`
4. Relational operators `<`, `<=` | `>` | `>=` | `=` | `<>` | `!=`
5. `+` | `-`
6. `*` | `/` | `div` | `mod`
7. Unary operators `-` | `not`
8. `if then else`
9. `prev` | `next` | `Get_Property`
10. `ID` | `QID` | `NestedDotID` | `Literal` | `pre` | `this` | `()`

Example: The two are equivalent.

```
x + if y then a else b * prev(z.f - 1, 0)
```

```
x + (if y then a else (b * (prev((z.f) - 1, 0))))
```

*Enumerator Expressions

When referencing enumeration members, enumerations defined locally in AGREE can use member names directly, while enumerations defined in AADL require the `enum()` syntax with the complete type path and member name specified.

Example code:

```
enum(Package::Color, Red)
```

Arrow Expression

Used to define the value rules of variables in the initial time step and subsequent time steps, serving as the initialization operator for streams.

`pre` cannot be used alone and must be paired with an arrow expression to initialize the initial step value.

Example code:

```
eq count: int = 0 -> pre(count) + 1;
```

`prev` Expression

Encapsulation of "arrow expression + `pre`", avoiding the risk of directly using `pre`.

Takes the second parameter "initial value" in the initial time step (step 1).

Takes the value of the first parameter "expression" from the previous step (i.e., `pre(expression)`) in all subsequent time steps (steps 2, 3, ...).

Example code:

```
eq x: int = prev(y + 1, 0);  -- x tracks the previous value of y + 1
```

`=>` Expression

Derivation relationship between two boolean propositions.

`A => B` means: If A, then B.

```
A => B ≡ (not A) or B
```

Truth table:

Premise A (Antecedent)	Conclusion B (Consequent)	Result of A => B (If A, then B)
true	true	true
true	false	false
false	true	true
false	false	true

`floor` Expression

Takes a real-type expression as a parameter and returns the floor result of the real number, with an integer type.

Example code:

```
eq real_num: real = 7.9;          -- Define a real variable
eq int_num: int = floor(real_num); -- The value of int_num is 7 (floor of 7.9)
```

`Real` Expression

Takes an integer-type expression as a parameter and returns a real number equal to the integer value.

Example code:

```
eq int_num: int = 4;              -- Define an integer variable
eq real_num: real = real(int_num); -- Convert integer 4 to a real number
eq result: real = real_num / 2.0;  -- Real number operation: 4.0 / 2.0 = 2.0
```

Get Property Expressions

A core expression in AGREE used to read property values in the AADL model, allowing users to reference properties of AADL components or objects in constraint logic.

AGREE only supports `bool`, `int`, and `real` type properties.

Basic syntax:

```
Get_Property(Expr, AADL_Property)
```

First parameter: "Relative path" of the object to which the property belongs.

- If the property belongs to the current component (i.e., the component where Get_Property is located), use `this` directly;
- If the property belongs to another component (such as a subcomponent, external component), write the "relative path" of the component in the instance model (in `NestedDotID` format, such as `subcomponent_name.grandchild_component_name`).

Second parameter: A property already defined in the AADL model.

For properties associated with specific values, AGREE directly uses the property value during analysis without additional processing.

For properties not associated with specific values, it automatically generates implicit constraints: the property value remains unchanged throughout the time steps of the analysis.